

01_intro

September 27, 2026

1 MA3622: Practical Machine Learning

variationalform https://variationalform.github.io/courses_PML/

Just Enough: progress at pace <https://github.com/variationalform>

Simon Shaw - <https://www.brunel.ac.uk/people/simon-shaw>.

1.1 What this module is about:

You will be introduced to ...

- fundamental ideas in machine learning, data science, and artificial intelligence:
 - k -NN: k -Nearest Neighbours;
 - perceptrons, neural networks and deep learning.
- underpinning mathematical concepts
- python implementations

1.2 Assessment

- 25% coursework (details to follow in a few weeks)
- 75% examination (revision and reflection time will be allocated)

1.3 Study Habits - a discussion

The Quality Assurance Agency for Higher Education (QAA, <https://www.qaa.ac.uk>) defines one academic credit as nominally equal to 10 hours of study (see <https://www.qaa.ac.uk/docs/qaa/quality-code/higher-education-credit-framework-for-england.pdf>).

Therefore, this 15 credit block requires nominally 150 hours of your time. **Although every one of us is different and may choose to spend our time in different ways**, the following sketch of these 150 hours is worth considering.

There will be 40 hours spent on two lectures plus two seminars/labs in each of ten weeks (weeks 1-5, 7-11 with reading week in week 6). There will be a two hour exam, to which you could assign 28 hours of preparation/revision time. This accounts for $40 + 2 + 28 = 70$ hours.

In addition there is assignment which you could allocate 20 hours to, making up to 90 hours. This leaves 60 of the 150 hours over. In each of 10 weeks of term there will be a requirement to engage in set tasks and problems, and to read sections of set books and sources in order to strengthen your

understanding of imparted material as well as to prepare you for the next topics. These 60 hours average out to 6 hours per week over those 10 weeks.

Note that engaging at this level does not guarantee any outcome, whether that be a bare pass or an A grade. It is a guideline only. If despite engaging at this level you are struggling to progress and achieve in the module then seek help and advice.

Further, these *weekly study hours* have to be **high quality inquisitive engagement**. Writing and re-writing notes, procrastinating, passively reading AI outputs and looking at but not engaging with learning materials don't really count.

You'll know when you're actually **working** - you'll feel it. Have a read of this [https://en.wikipedia.org/wiki/Flow_\(psychology\)](https://en.wikipedia.org/wiki/Flow_(psychology)) and make learning a daily rewarding habit.

2 Practical Machine Learning

2.1 Introduction to key concepts

- fundamental ideas in machine learning, data science, and artificial intelligence:
- Leading to... k -NN: k -Nearest Neighbours and Logistic Regression
- underpinning mathematical concepts and **python** implementation with **sklearn**

variationalform: https://variationalform.github.io/courses_PML/

Just Enough: progress at pace <https://github.com/variationalform>

This work is licensed under CC BY-SA 4.0 (Attribution-ShareAlike 4.0 International)

Visit <http://creativecommons.org/licenses/by-sa/4.0/> to see the terms.

This document uses python

and also makes use of LaTeX

in Markdown

Google and possibly other search engines, both with and without AI enhancement, may have been used in the production of these notes.

AI tools such as Gemini, Claude, ChaptGPT etc may also have played a role in generating ideas and raw material for the following content.

Specific inclusions and sources are always referenced.

However, all planning and quality control is purely human, and my responsibility.

2.2 Key Concepts: Glossary of Relevant Terms

The first few of these are debateable, evolving and subject to change and interpretation. It's worth searching and reading for yourself. These are a fast growing areas.

Data Science A blend of mathematics, computer science and statistics brought to bear with some form of domain expertise.

Data Analytics Systematic computational analysis of data, used typically to discover value and insights.

Data Engineering The stewardship, cleaning, warehousing and preparation of data to support its pipelining to its exploitation.

Artificial Intelligence (AI) The development and deployment of digital systems that can effectively substitute for humans in tasks beyond the routine application of fixed rules. When you talk to your home assistant, your phone, or your satellite TV receiver, or your car, or your laptop, and so on, it has no idea what you are going to say. It doesn't have a bank of pre-answered questions, but instead it responds dynamically to what it hears. It has been trained on data, and it has learned how to respond. Incidentally, how do you think these systems even understand what you said? As a child, it took you months to begin to understand human speech...

Machine Learning (ML) The development and deployment of algorithms that are able to learn from data without explicit instructions, and then analyze, predict, or otherwise draw inferences, from unseen data. These algorithms would typically be expected to add measurable value by their performance.

*Consider for example an algorithm that predicted **tails** for every coin flip. It's right half the time - but there's no value in that.*

ML is a big part of AI but not all of it. AI has historically drawn on many techniques, but in current times AI is becoming more and more based on ML techniques.

We'll often refer to ML/AI in these notes.

Learning Machine learning models do not have intrinsic knowledge but instead learn from data. Typically a data set comprises a list of items each of which has one or more *features* which correspond to a *label*. We'll see some examples of this below.

We think of the features as being inputs to the machine learning model, and the label as being the output. Typically we want to be able to feed in new features, and have the model predict the label.

To do this we need a **training data set** so that the model can learn how to map the features to the label: the *input to the output*.

There are three basic learning paradigms:

- **Supervised Learning:** Here the data is labelled. This means that for a given set of features, or inputs, we also know their labels, or outputs. Examples of this are where...
 - We could have a list of features of insured drivers, such as age, time since they passed their driving test, type of car, locality, and along with those features a monetary value on their accident claim. The task would be to learn how much of an insurance premium to charge to a new customer once those features have been determined.
 - We might have a bank of images of handwritten digits, and for each image we know what digit is represented. The MNIST database of handwritten digits, see <http://yann.lecun.com/exdb/mnist/> or https://en.wikipedia.org/wiki/MNIST_database for example, is a well known example of this. The task is to learn how to predict what digit is captured by a new image. This could be used in ANPR systems for example, https://en.wikipedia.org/wiki/Automatic_number-plate_recognition.

- **Unsupervised Learning:** This is where we only know the features and we want to cluster the data in such a way that a set of similar features can be associated with some common characteristic (the label).
 - This can be used on data where the analyst doesn't initially know what they are looking for. For example, a retailer might have a mass of data of customer age, locale, average spend, types of purchased item, time of day of purchase, day of week of purchase, time of year etc. What characteristics can be used to group these customers? How can advertising be targetted?
 - principal component analysis seeks to re-orient data so that its dominant statistical properties are revealed (this is covered in a different module).
- **Reinforcement Learning:** This seeks to strike a balance between the two above. There are no labels, but instead, as time progresses the learning algorithm has a *reward* variable which is increased when an action it has learned has resulted in a measurable benefit. Over time the algorithm develops a policy to inform its actions.

Our module is concerned mainly with **supervised learning**.

Regression and Classification Machine Learning algorithms usually perform one of the following tasks:

- **Regression:** here the output, the label, can take any value in a continuous set. For example, the height of a tree, given local climate, soil type, genus, age since planting, could be considered to be any non-negative real number (although not with equal probability).
- **Classification:** in this case the label will be deemed to be one of a certain class. For example, in the handwritten digits example above, the output will be one of the digits $\{0, 1, 2, 3, \dots, 9\}$.

Some algorithms are able to perform both the regression and classification tasks, some are specialized to just one task.

2.3 Reading List

Our main sources of information are as follows:

- MML: Mathematics for Machine Learning, by Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. Cambridge University Press. <https://mml-book.github.io>.
- MLF: Machine Learning: A First Course for Engineers and Scientists, by Andreas Lindholm, Niklas Wahlström, Fredrik Lindsten, Thomas B. Schön. Cambridge University Press. <http://smlbook.org>.
- MLF2 - as above but the second edition also linked at <http://smlbook.org>
- UDL: Understanding Deep Learning, by S.J.D.Prince. MIT Press, 2023. <https://udlbook.github.io/udlbook>

All of the above can be accessed legally and without cost.

There are also these useful references for coding:

- PT: python: <https://docs.python.org/3/tutorial>
- NP: numpy: <https://numpy.org/doc/stable/user/quickstart.html>
- MPL: matplotlib: <https://matplotlib.org>

The capitalized abbreviations will be used throughout to refer to these sources. For example, we could say *See [MLF, Chap 2, Sec. 1] for more discussion of **Supervised Learning***. This would just be a quick way of saying

Look in Section 1, of Chapter 2, of the first edition of Machine Learning: A First Course for Engineers and Scientists, by Andreas Lindholm, Niklas Wahlström, Fredrik Lindsten, Thomas B. Schön, for more discussion of supervised learning.

There may be other sources shared as we go along. For now these will get us a long way.

2.4 Coding: python and some data sets

We use **python** because its use in commercial and academic ML/AI seems to be pre-eminent.

much of our code is implemented in well-known and well-documented **python** libraries. These are the main ones we will use:

- **matplotlib**: used to create visualizations, plotting 2D graphs in particular.
- **numpy**: this is *numerical python*, it is used for array processing which for us will usually mean the numerical calculations involving vectors and matrices.
- **scikit-learn**: a set of well documented and easy to use tools for predictive data analysis.
- **pandas**: a data analysis tool, used for the storing and manipulation of data.
- **seaborn**: a data visualization library for attractive and informative statistical graphics.

There will be others, but these are the main ones.

2.5 Binder, Anaconda, Jupyter - a first look at some data

Eventually we will use the anaconda distribution to access **python** and the libraries we need. The coding itself will be carried out in a Jupyter notebook. We'll go through this in an early lab session. We'll start though with Binder: click here:

<https://mybinder.org/v2/gh/variationalform/PML.git/HEAD>

Let's see some code and some data. In the following cell we import **seaborn** and look at the names of the built in data sets. The **seaborn** library, <https://seaborn.pydata.org>, is designed for data visualization. It uses **matplotlib**, <https://matplotlib.org>, which is a graphics library for **python**.

If you want to dig deeper, you can look at <https://blog.enterprisedna.co/how-to-load-sample-datasets-in-python/> and <https://github.com/mwaskom/seaborn-data> for the background - but you don't need to.

```
[1]: import seaborn as sns
# we can now refer to the seaborn library functions using 'sns'
# you could use another character string, but 'sns' is standard.

# note that # is used to write 'comments'
# Now let's get the names of the built-in data sets.
datasets = sns.get_dataset_names()
for dataset in datasets:
    print(dataset, end=', ')
```

```
# type SHIFT=RETURN to execute the highlighted (active) cell
```

anagrams, anscombe, attention, brain_networks, car_crashes, diamonds, dots, dowjones, exercise, flights, fmri, geyser, glue, healthexp, iris, mpg, penguins, planets, seaice, taxis, tips, titanic,

2.5.1 The taxis data set

Below, the variable `dft` is a pandas data frame: `dft = 'data frame taxis'`

```
[2]: # let's take a look at 'taxis'
dft = sns.load_dataset('taxis')
# this just plots the first few lines of the data
dft.head()
```

```
[2]:
```

		pickup		dropoff	passengers	distance	fare	tip	\
0	2019-03-23	20:21:09	2019-03-23	20:27:24	1	1.60	7.0	2.15	
1	2019-03-04	16:11:55	2019-03-04	16:19:00	1	0.79	5.0	0.00	
2	2019-03-27	17:53:01	2019-03-27	18:00:25	1	1.37	7.5	2.36	
3	2019-03-10	01:23:59	2019-03-10	01:49:51	1	7.70	27.0	6.15	
4	2019-03-30	13:27:42	2019-03-30	13:37:14	3	2.16	9.0	1.10	

		tolls	total	color	payment	pickup_zone	\
0		0.0	12.95	yellow	credit card	Lenox Hill West	
1		0.0	9.30	yellow	cash	Upper West Side South	
2		0.0	14.16	yellow	credit card	Alphabet City	
3		0.0	36.95	yellow	credit card	Hudson Sq	
4		0.0	13.40	yellow	credit card	Midtown East	

		dropoff_zone	pickup_borough	dropoff_borough
0		UN/Turtle Bay South	Manhattan	Manhattan
1		Upper West Side South	Manhattan	Manhattan
2		West Village	Manhattan	Manhattan
3		Yorkville West	Manhattan	Manhattan
4		Yorkville West	Manhattan	Manhattan

```
[3]: # this will show the last two lines... There are 6433 records (Why?)
dft.tail(2)
```

```
[3]:
```

		pickup		dropoff	passengers	distance	fare	\
6431	2019-03-04	10:09:25	2019-03-04	10:14:29	1	1.12	6.0	
6432	2019-03-13	19:31:22	2019-03-13	19:48:02	1	3.85	15.0	

		tip	tolls	total	color	payment	pickup_zone	\
6431		0.00	0.0	6.80	green	credit card	East New York	
6432		3.36	0.0	20.16	green	credit card	Boerum Hill	

		dropoff_zone	pickup_borough	dropoff_borough
--	--	--------------	----------------	-----------------

6431	East Flatbush/Remsen Village	Brooklyn	Brooklyn
6432	Windsor Terrace	Brooklyn	Brooklyn

What we are seeing here is a **data frame**. It is furnished by the **pandas** library: <https://pandas.pydata.org> which is used by the **seaborn** library to store its example data sets. Each row of the data frame corresponds to a single **data point**, which we could also call an **observation** or **measurement** (depending on context).

Each column (except the left-most) corresponds to a **feature** of the data point. The first column is just an index giving the row number. Note that this index starts at zero - so, for example, the third row will be labelled/indexed as 2. Be careful of this - it can be confusing.

```
[4]: # let's print the data frame...
print(dft)
```

		pickup	dropoff	passengers	distance	fare \
0	2019-03-23	20:21:09	2019-03-23 20:27:24	1	1.60	7.0
1	2019-03-04	16:11:55	2019-03-04 16:19:00	1	0.79	5.0
2	2019-03-27	17:53:01	2019-03-27 18:00:25	1	1.37	7.5
3	2019-03-10	01:23:59	2019-03-10 01:49:51	1	7.70	27.0
4	2019-03-30	13:27:42	2019-03-30 13:37:14	3	2.16	9.0
...	...	...	...	...	...	...
6428	2019-03-31	09:51:53	2019-03-31 09:55:27	1	0.75	4.5
6429	2019-03-31	17:38:00	2019-03-31 18:34:23	1	18.74	58.0
6430	2019-03-23	22:55:18	2019-03-23 23:14:25	1	4.14	16.0
6431	2019-03-04	10:09:25	2019-03-04 10:14:29	1	1.12	6.0
6432	2019-03-13	19:31:22	2019-03-13 19:48:02	1	3.85	15.0

	tip	tolls	total	color	payment	pickup_zone \
0	2.15	0.0	12.95	yellow	credit card	Lenox Hill West
1	0.00	0.0	9.30	yellow	cash	Upper West Side South
2	2.36	0.0	14.16	yellow	credit card	Alphabet City
3	6.15	0.0	36.95	yellow	credit card	Hudson Sq
4	1.10	0.0	13.40	yellow	credit card	Midtown East
...	...	...	...	...	...	...
6428	1.06	0.0	6.36	green	credit card	East Harlem North
6429	0.00	0.0	58.80	green	credit card	Jamaica
6430	0.00	0.0	17.30	green	cash	Crown Heights North
6431	0.00	0.0	6.80	green	credit card	East New York
6432	3.36	0.0	20.16	green	credit card	Boerum Hill

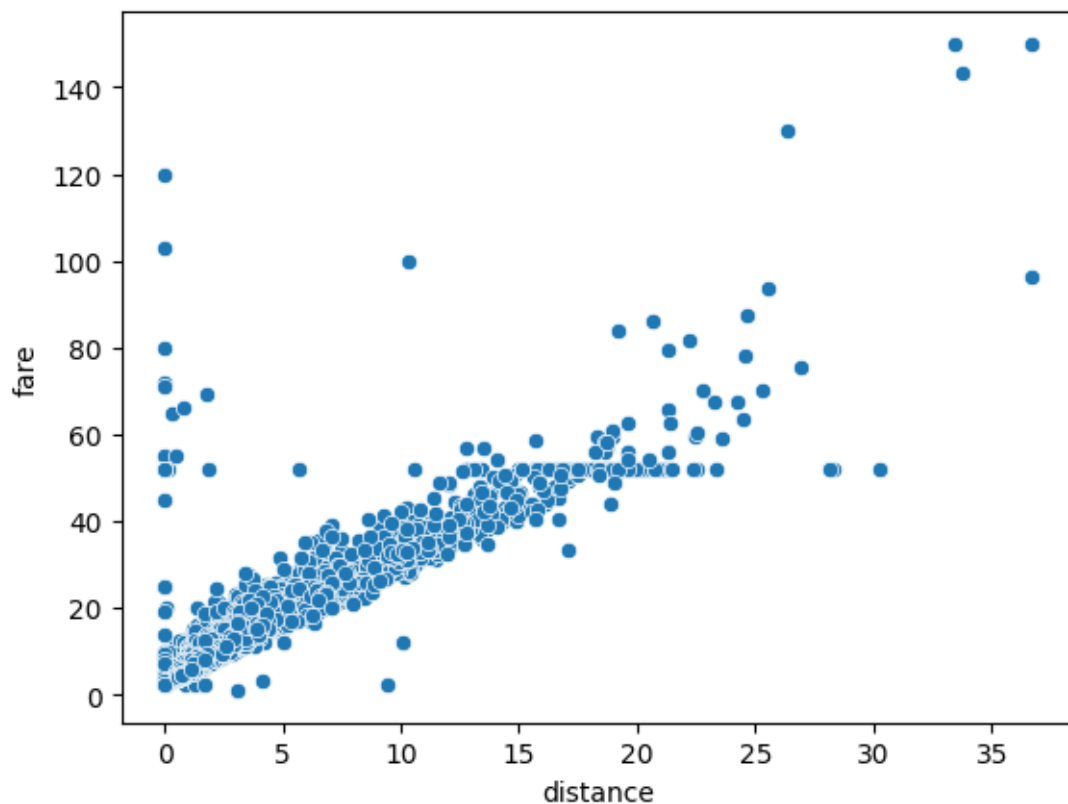
	dropoff_zone	pickup_borough	dropoff_borough
0	UN/Turtle Bay South	Manhattan	Manhattan
1	Upper West Side South	Manhattan	Manhattan
2	West Village	Manhattan	Manhattan
3	Yorkville West	Manhattan	Manhattan
4	Yorkville West	Manhattan	Manhattan
...	...	...	...

6428	Central Harlem North	Manhattan	Manhattan
6429	East Concourse/Concourse Village	Queens	Bronx
6430	Bushwick North	Brooklyn	Brooklyn
6431	East Flatbush/Remsen Village	Brooklyn	Brooklyn
6432	Windsor Terrace	Brooklyn	Brooklyn

[6433 rows x 14 columns]

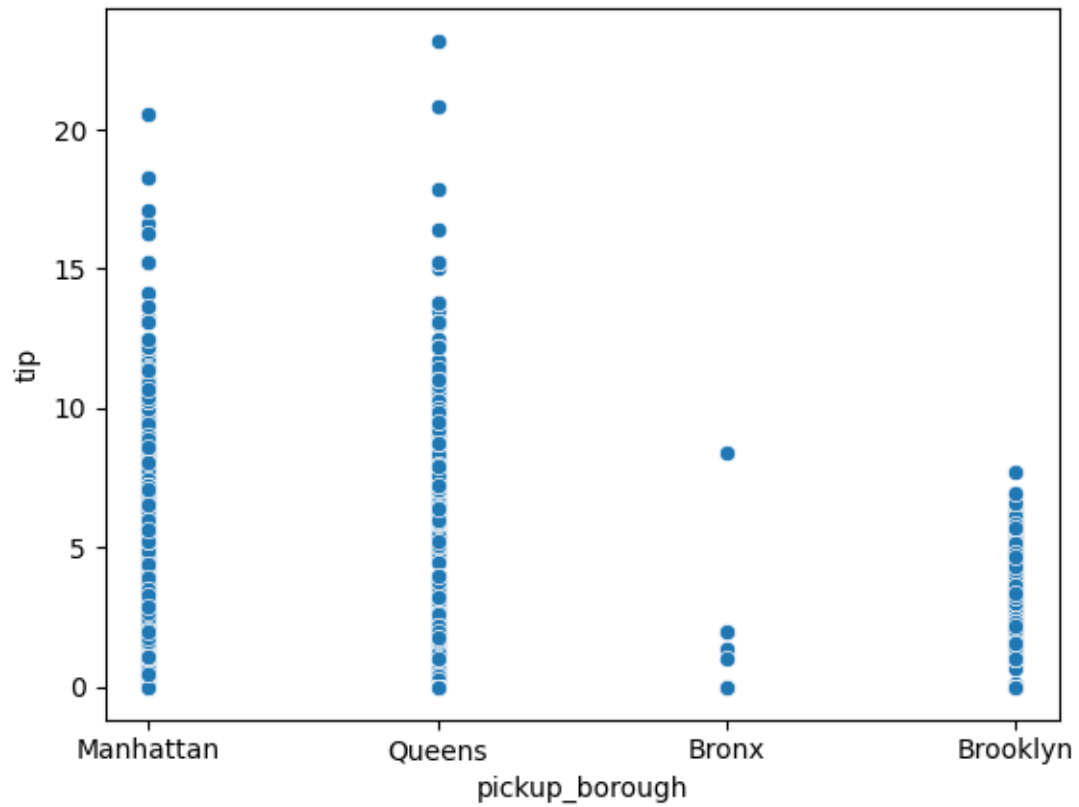
Visualization What a mess! Rows and rows of numbers aren't that helpful. `seaborn` makes visualization easy - here is a scatter plot of the data.

```
[5]: sns.scatterplot(data=dft, x="distance", y="fare");
```

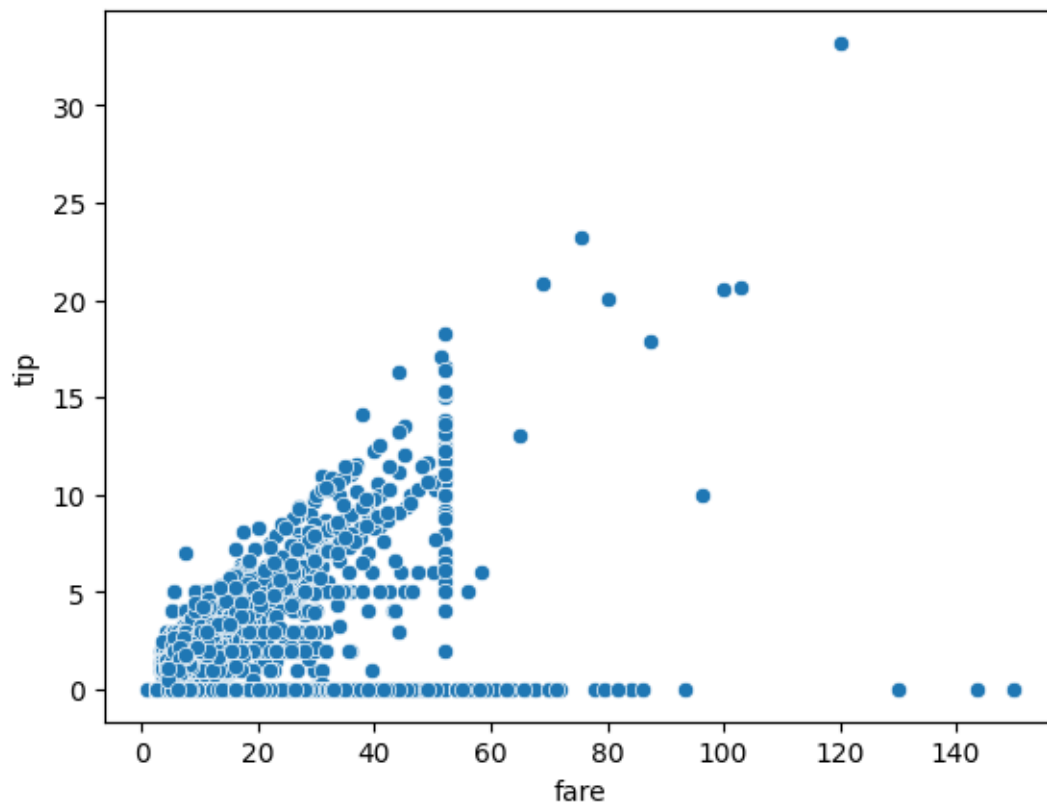


THINK ABOUT: it looks like fare is roughly proportional to distance. But what could cause the outliers?

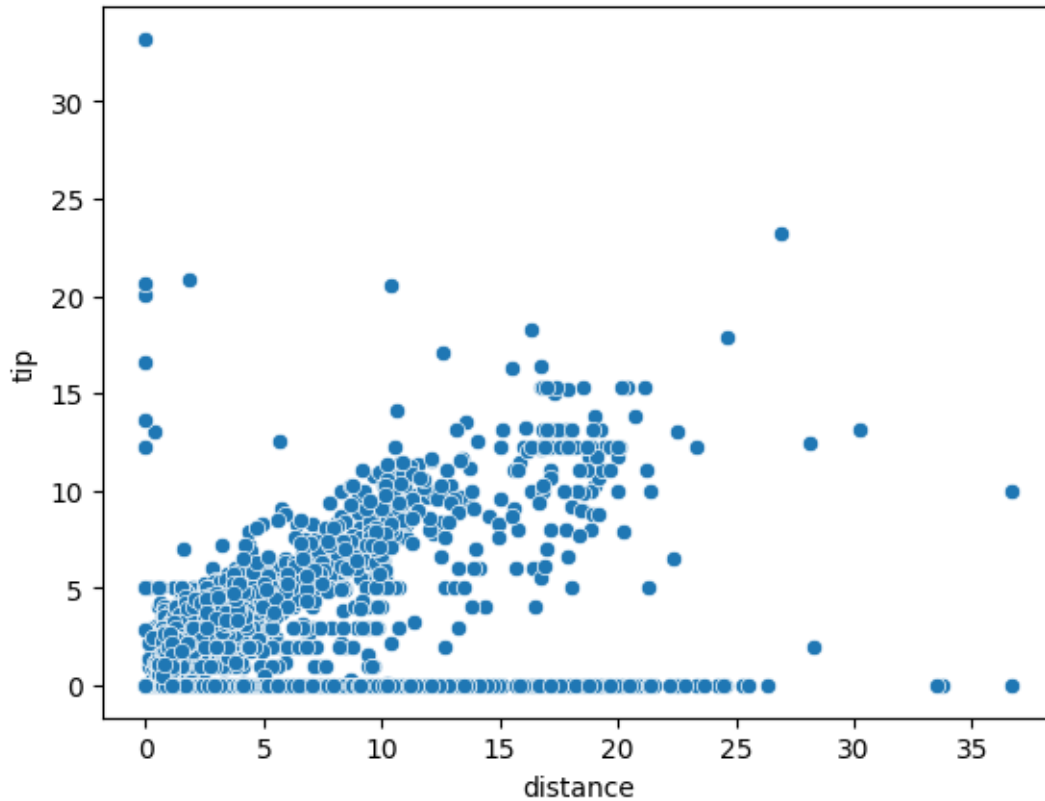
```
[6]: # here's another example
sns.scatterplot(data=dft, x="pickup_borough", y="tip");
```



```
[7]: # is the tip proportional to the fare?  
sns.scatterplot(data=dft, x="fare", y="tip");
```



```
[8]: # is the tip proportional to the distance?  
sns.scatterplot(data=dft, x="distance", y="tip");
```



2.6 Exercises

For the `taxi` data set:

1. Produce a scatterplot of “dropoff_borough” vs. “tip”
2. Plot the dependence of fare on distance.

```
1: sns.scatterplot(data=ds, x="dropoff_borough", y="tip")
```

```
2: sns.scatterplot(data=ds, x="distance", y="tip")
```

3 Next Steps...

Let's look now at a fundamental ML algorithm...

***k*-NN: *k* Nearest Neighbours**

We'll move to the next set of slides.

3.1 Technical Notes, Production and Archiving

Ignore the material below. What follows is not relevant to the material being taught.

Production Workflow

- Finalise the notebook material above
- Set OUTPUTTING=1 below
- Clear and fresh run of entire notebook
- Create html slide show:
 - `jupyter nbconvert --to slides 1_intro.ipynb`
- Clear all cell output
- Set OUTPUTTING=0 below
- Save
- git add, commit and push to FML
- copy PDF, HTML etc to web site
 - git add, commit and push
- rebuild binder

Some of this originated from

<https://stackoverflow.com/questions/38540326/save-html-of-a-jupyter-notebook-from-within-the-r>

These lines create a back up of the notebook. They can be ignored.

At some point this is better as a bash script outside of the notebook

```
[10]: %%%bash
NBROOTNAME='01_intro'
OUTPUTTING=1

if [ $OUTPUTTING -eq 1 ]; then
  jupyter nbconvert --to slides $NBROOTNAME.ipynb
  cp $NBROOTNAME.slides.html ../backups/$(date +"%m_%d_%Y-%H%M%S")_$NBROOTNAME.
↪slides.html
  mv -f $NBROOTNAME.slides.html ../formats/slides/

  jupyter nbconvert --to pdf $NBROOTNAME.ipynb
  cp $NBROOTNAME.pdf ../backups/$(date +"%m_%d_%Y-%H%M%S")_$NBROOTNAME.pdf
  mv -f $NBROOTNAME.pdf ../formats/pdf/

  jupyter nbconvert --to script $NBROOTNAME.ipynb
  cp $NBROOTNAME.py ../backups/$(date +"%m_%d_%Y-%H%M%S")_$NBROOTNAME.py
  mv -f $NBROOTNAME.py ../formats/py/
  echo; echo 'Finished generating html, pdf and py output versions'
else
  echo 'Not generating html, pdf and py output versions'
fi
```

```
[NbConvertApp] Converting notebook 01_intro.ipynb to slides
```

```
[NbConvertApp] WARNING | Alternative text is missing on 6 image(s).
```

```
[NbConvertApp] Writing 308608 bytes to 01_intro.slides.html
```

```
[NbConvertApp] Converting notebook 01_intro.ipynb to pdf
```

```
/Users/simon/offline/research/ALCAS/Nilocas_repos/NilocasUV/lib/python3.14/site-
```

```
packages/nbconvert/utils/pandoc.py:50: RuntimeWarning: You are using an
unsupported version of pandoc (2.7.2).
Your version must be at least (2.9.2) but less than (4.0.0).
Refer to https://pandoc.org/installing.html.
Continuing with doubts...
    check_pandoc_version()
[NbConvertApp] Writing 45280 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 81997 bytes to 01_intro.pdf
[NbConvertApp] Converting notebook 01_intro.ipynb to script
[NbConvertApp] Writing 17943 bytes to 01_intro.py
```

Finished generating html, pdf and py output versions